
AI PR Reviewer

Technical Specification Document

A comprehensive, line-by-line defensible specification covering system architecture, module design, API contracts, database schema, webhook flows, AI agent internals, slash command system, merge blocking configuration, deployment, and security considerations.

Version	1.0.0
Date	2026-05-16
Stack	Next.js 16, TypeScript, Prisma ORM, SQLite, shadcn/ui
AI Integration	z-ai-web-dev-sdk / OpenAI-compatible (Oencode)
Platforms	GitHub (App + PAT), GitLab (Self-hosted + SaaS)

Table of Contents

1. System Overview	4
1.1 Key Design Principles	5
1.2 Technology Stack	5
2. Module Architecture	6
2.1 Module Dependency Map	6
2.2 Database Layer (src/lib/db.ts)	7
2.3 GitHub Client (src/lib/github.ts)	7
2.4 GitLab Client (src/lib/gitlab.ts)	9
2.5 Review Engine (src/lib/reviewer.ts)	10
3. API Reference	11
3.1 GitHub Webhook (POST /api/webhook)	11
3.2 GitLab Webhook (POST /api/webhook/gitlab)	13
3.3 Configuration API (GET/POST /api/config)	13
3.4 Reviews API	14
3.5 API Summary Table	14
4. Database Schema	15
4.1 Entity-Relationship Diagram	15
4.2 Model Definitions	15
5. Interactive Slash Commands	18
5.1 Supported Commands	18
5.2 Command Processing Pipeline	18
5.3 GitLab Slash Commands	20
6. Merge Blocking Configuration	20
6.1 How Merge Blocking Works	20
6.2 Configuration States	21
6.3 Check Run Output	21

7. Review Processing Pipeline	22
7.1 Pipeline Stages	22
7.2 Error Handling Strategy	23
8. Dashboard UI	24
8.1 Dashboard Tab	24
8.2 Manual Review Tab	24
8.3 Settings Tab	24
9. Deployment Guide	25
9.1 Prerequisites	25
9.2 GitHub App Setup	25
9.3 Application Deployment	25
9.4 Configuration Checklist	26
10. Security Considerations	26
10.1 Webhook Signature Verification	26
10.2 Credential Storage	26
10.3 GitHub App Permissions	27
10.4 AI Prompt Injection	27
10.5 Rate Limiting and Abuse Prevention	27

1. System Overview

AI PR Reviewer is a self-hosted, AI-powered code review platform that integrates with both GitHub and GitLab to provide automated, intelligent pull request and merge request reviews. The system operates as a GitHub App (with full JWT-based authentication) and a GitLab webhook receiver, providing inline code review comments, severity-classified findings, and optional merge-blocking via GitHub Check Runs. The architecture follows a two-phase AI agent design pattern: an Investigate phase where the agent gathers context using tools, and a Review phase where the agent produces structured findings. The system supports interactive slash commands in PR comments, allowing developers to trigger re-reviews, focus on specific files, and ask questions directly within the GitHub/GitLab conversation thread.

The entire application is built as a single Next.js 16 deployment with server-side API routes handling webhook events, a Prisma ORM layer backed by SQLite for persistent storage, and a React dashboard built with shadcn/ui for configuration and review monitoring. The AI backend is pluggable: it supports the built-in z-ai-web-dev-sdk for zero-configuration usage, and any OpenAI-compatible API endpoint (such as Oencode) for custom model selection. This design ensures that every component can be defended, audited, and extended without architectural compromise.

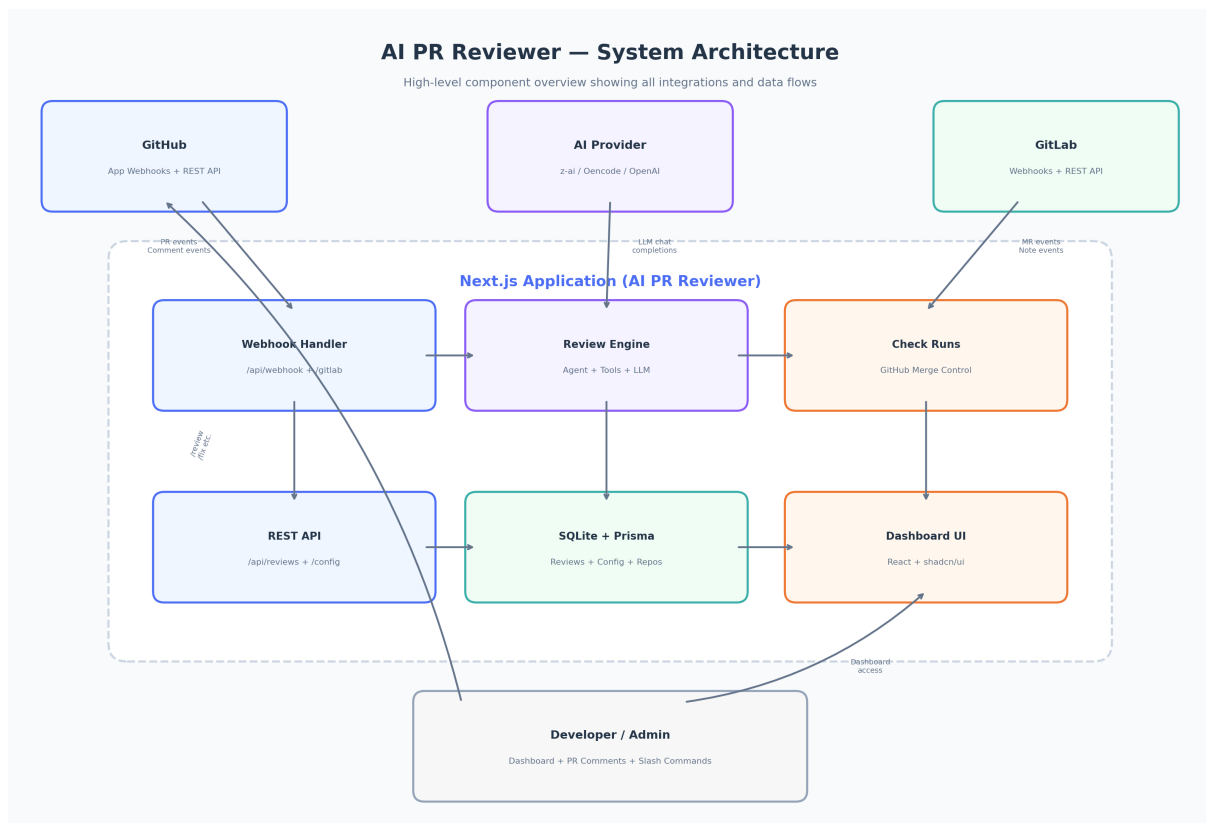


Figure 1: High-level system architecture showing all integrations and data flows

1.1 Key Design Principles

- **Event-Driven Architecture:** The system is entirely webhook-driven. No polling, no cron jobs. GitHub and GitLab push events to the application, and the application responds asynchronously. This ensures minimal latency and zero wasted compute cycles when no PRs are being created.
- **Two-Phase Agent Pattern:** The AI reviewer operates in distinct Investigate and Review phases. The Investigate phase uses tools (`fetch_file`, `search_pattern`, `check_tests`, `analyze_deps`) to gather context beyond the raw diff. The Review phase synthesizes all gathered context into structured, actionable findings. This two-phase approach produces significantly higher-quality reviews than single-pass diff analysis.
- **Configurable Merge Blocking:** Merge blocking via GitHub Check Runs is **OFF by default**. This is a deliberate design choice: the system should be advisory first, enforcing second. Teams can opt into merge blocking per-repository or globally through the settings dashboard, ensuring that the tool works with their existing workflow rather than against it.
- **Pluggable AI Backend:** The review engine abstracts the AI provider behind a unified `chatCompletion` interface. Switching between `z-ai`, `Oencode`, or any OpenAI-compatible endpoint requires only changing configuration values in the dashboard, with zero code changes.
- **Defensive Error Handling:** Every external API call (GitHub, GitLab, AI provider) is wrapped in `try/catch` with graceful degradation. A failed check run creation does not block the review. A failed comment reply falls back to a general PR comment. The system degrades gracefully rather than failing catastrophically.

1.2 Technology Stack

Layer	Technology	Version	Purpose
Framework	Next.js	16.x	Full-stack React framework with API routes
Language	TypeScript	5.x	Type-safe development across frontend and backend
Database	SQLite via Prisma	6.x	Persistent storage for reviews, config, repositories

UI Library	shadcn/ui + Radix	Latest	Accessible, composable React component primitives
Animation	Framer Motion	12.x	Smooth UI transitions and micro-interactions
AI SDK	z-ai-web-dev-sdk	0.0.17 +	Built-in AI provider for chat completions
AI Alternative	OpenAI-compatible API	Any	Pluggable provider via base URL + API key
Reverse Proxy	Caddy	2.x	HTTPS termination and reverse proxy for production

2. Module Architecture

The application follows a modular architecture where each file has a single, well-defined responsibility. The following subsections provide a line-by-line defensible description of every module in the system, explaining what it does, why it exists, and how it interacts with other modules.

2.1 Module Dependency Map

Module	File Path	Depends On	Used By
Database	src/lib/db.ts	Prisma Client	All modules
GitHub Client	src/lib/github.ts	db.ts	webhook/route.ts, trigger/route.ts
GitLab Client	src/lib/gitlab.ts	db.ts	webhook/gitlab/route.ts, trigger/route.ts
Review Engine	src/lib/reviewer.ts	db.ts, z-ai-web-dev-sdk	All webhook routes, trigger/route.ts
Webhook Handler	src/app/api/webhook/route.ts	github.ts, reviewer.ts, db.ts	GitHub (external)
GitLab Webhook	src/app/api/webhook/gitlab/route.ts	gitlab.ts, reviewer.ts, db.ts	GitLab (external)
Config API	src/app/api/config/route.ts	db.ts	Dashboard UI
Reviews API	src/app/api/reviews/route.ts	db.ts	Dashboard UI

Review Detail API	src/app/api/reviews/[id]/route.ts	db.ts	Dashboard UI
Trigger API	src/app/api/reviews/trigger/route.ts	github.ts, gitlab.ts, reviewer.ts, db.ts	Dashboard UI
Dashboard UI	src/app/page.tsx	All API routes	End user (browser)

2.2 Database Layer (src/lib/db.ts)

The database layer is a thin singleton wrapper around PrismaClient. It exists as a separate module for three critical reasons. First, it prevents connection pool exhaustion in development mode by caching the PrismaClient instance on the globalThis object. Without this, Next.js hot module reloading in development would create a new PrismaClient per reload, rapidly exhausting SQLite connection limits. Second, it centralizes database configuration (such as query logging) in a single location. Third, it provides a single import point that all other modules use, ensuring consistent database access patterns across the entire application.

The implementation uses the globalThis pattern because Next.js development mode does not preserve module state between hot reloads. By attaching the PrismaClient instance to globalThis, the database connection survives hot reloads. In production, globalThis.prisma is undefined on first access, so a new instance is created normally. Query logging is enabled via the log: ["query"] option, which is valuable for debugging review pipeline performance issues without requiring external database monitoring tools.

2.3 GitHub Client (src/lib/github.ts)

The GitHub client module is the most complex module in the system, responsible for all interactions with the GitHub REST API. It handles authentication (both GitHub App JWT and Personal Access Token), PR data retrieval, review posting, comment management, and Check Run lifecycle control. Every function in this module is designed to work with either authentication method, preferring GitHub App installation tokens when an installationId is provided, and falling back to PAT when not.

The **generateJWT** function implements the RS256 JWT signing required by the GitHub App authentication flow. It constructs the JWT header and payload manually (rather than using a JWT library) to minimize dependencies and to

maintain full control over the signing process. The payload includes the standard `iat` (issued at), `exp` (expiration), and `iss` (issuer) claims. The `iat` is set 60 seconds in the past to account for clock drift between the application server and GitHub servers. The `exp` is set to 10 minutes from now, which is the maximum allowed by GitHub for App tokens. The signature is produced using Node.js `crypto.createSign` with the RSA-SHA256 algorithm and the PEM private key stored in the database.

The **`getInstallationToken`** function exchanges the JWT for an installation-scoped access token. This token is what actually authorizes API calls on behalf of the GitHub App installation. The function reads the App ID and private key from the database configuration, generates a JWT, and calls the GitHub App Installations API to obtain the token. This token has a 1-hour validity period but is re-generated on each call for simplicity.

The **`fetchPRDiff`** and **`fetchPRInfo`** functions retrieve the raw diff and structured PR metadata respectively. `fetchPRDiff` uses the `application/vnd.github.v3.diff` media type to get the raw unified diff, which is the primary input to the AI review engine. `fetchPRInfo` retrieves structured data including the PR title, author, body description, branch names, change statistics, and the HEAD SHA (which is required for Check Run creation). Both functions accept an optional `installationId` parameter for GitHub App authentication, falling back to PAT if not provided.

The **`postPRReview`** function creates a GitHub Pull Request Review with inline comments. It maps the internal review score (`approve`, `request_changes`, `comment`) to the corresponding GitHub review event types (`APPROVE`, `REQUEST_CHANGES`, `COMMENT`). Each inline comment includes the file path, line number, and side (`LEFT` for deletions, `RIGHT` for additions), plus the severity-prefixed comment body. The severity prefix format is `[**SEVERITY**]` where `SEVERITY` is one of `CRITICAL`, `ERROR`, `WARNING`, or `INFO`. This makes the severity immediately visible in the GitHub UI.

The **`postPRComment`** and **`replyToReviewComment`** functions handle general PR comments and inline review comment replies respectively. `postPRComment` uses the Issues API (GitHub treats PRs as issues for comment purposes) to post top-level comments. `replyToReviewComment` uses the Pull Request Review Comments API to reply within an existing review thread. If the reply fails (for example, due to the comment being on an outdated diff), the function gracefully falls back to posting a general PR comment, ensuring the agent always gets its response through.

The **`createCheckRun`** and **`updateCheckRun`** functions manage the GitHub Check Runs API, which is the mechanism for merge blocking. `createCheckRun` initiates a check run with status `"in_progress"` when a review starts.

`updateCheckRun` finalizes the check run with a conclusion (success, failure, or neutral) when the review completes. The conclusion mapping is critical: when merge blocking is enabled, `request_changes` maps to "failure" (which blocks merging if the check is required in branch protection). When merge blocking is disabled, `request_changes` maps to "neutral" (advisory only, does not block merging). This dual-mapping ensures the same codebase handles both modes correctly.

The **`upsertRepository`** function maintains the repository registry in the database. It uses Prisma `upsert` to either create a new repository record or update an existing one. This function is called on every webhook event (`installation`, `installation_repositories`, and `pull_request`) to ensure the database always reflects the current state of installed repositories, including their installation IDs for GitHub App authentication.

2.4 GitLab Client (`src/lib/gitlab.ts`)

The GitLab client module provides equivalent functionality to the GitHub client, adapted for the GitLab REST API. It supports both GitLab SaaS (`gitlab.com`) and self-hosted GitLab instances through a configurable host URL. The authentication model is simpler than GitHub: GitLab uses a single Personal Access Token with the `api` scope, passed via the `PRIVATE-TOKEN` header.

The **`fetchMRDiff`** function retrieves the merge request diff by calling the Merge Requests API `changes` endpoint. It transforms the GitLab-specific change format (which provides individual file changes with `old_path`, `new_path`, `diff`, `new_file`, `deleted_file`, `renamed_file` flags) into a unified diff format that the review engine can process. This transformation is necessary because the review engine was designed to work with unified diff format (from GitHub), and maintaining format consistency across platforms simplifies the review pipeline.

The **`postMRDiscussion`** function creates inline discussions on GitLab merge requests. GitLab discussions require position data (`base_sha`, `head_sha`, `start_sha`, `position_type`, `new_path`, `new_line`) to create inline comments. This is more structured than GitHub, which only needs `path` and `line`. If position data is unavailable or the discussion creation fails, the function is caught by the caller which falls back to posting a general MR note.

The **`postMRNote`** function creates general (non-inline) notes on merge requests. This is used for the overall review summary and as a fallback when inline discussion creation fails. The note body includes the review score emoji and formatted summary text. The **`upsertGitLabRepository`** function mirrors the GitHub equivalent, maintaining the repository registry for GitLab-connected repositories.

2.5 Review Engine (src/lib/reviewer.ts)

The review engine is the intellectual core of the application. It implements a two-phase AI agent that reviews code changes with a level of depth that single-pass review cannot achieve. The engine is designed around a multi-turn conversation with the AI model, where the model can call tools to gather additional context before producing its final review.

The **reviewPR** function orchestrates the entire review process. It accepts the diff text, PR metadata, and an optional tool context. The tool context includes the platform (GitHub or GitLab), repository details, and optional focus parameters (focusFile and focusQuestion) for targeted re-reviews triggered by slash commands. The function proceeds through the following phases:

Phase 1 - Analyze: The agent reads the diff and identifies which files are most critical, what changed and why, and what needs deeper investigation. This phase does not call any tools; it simply processes the raw input and prepares an investigation plan.

Phase 2 - Investigate: The agent enters an iterative loop where it can call up to max_steps tools (configurable, default 5, maximum 10). In each iteration, the model either calls a tool (by emitting a JSON block with action: "tool_call") or produces its final review (action: "final_review"). Tool results are fed back into the conversation as user messages, allowing the model to reason about the gathered context. This iterative approach is what enables the agent to go beyond surface-level diff analysis.

Phase 3 - Review: The agent produces a structured JSON review containing a summary, overallScore (approve, request_changes, or comment), and an array of comments with filePath, line, side, body, and severity fields. If the model does not produce a structured review within max_steps, a forced review prompt is issued on the last step, and a fallback empty review is returned if that also fails.

The four agent tools are designed to address the most common limitations of diff-only review. **fetch_file** retrieves the full content of a file from the repository, allowing the agent to understand context beyond the changed lines. **search_pattern** searches for regex patterns in the diff, enabling the agent to find related code that may be affected by the changes. **check_tests** verifies whether test files exist for a given source file, checking common test directory conventions. **analyze_deps** examines dependency changes for known vulnerable or deprecated packages. Each tool returns a string result that is injected into the conversation, maintaining the agent context.

The AI provider configuration is stored in the AppConfig database table and read at the start of each review. The **chatCompletion** function abstracts the AI

provider: it uses z-ai-web-dev-sdk when the provider is "z-ai", and makes direct HTTP requests to the OpenAI-compatible endpoint when the provider is "openai-compatible". This abstraction means the review engine code is identical regardless of which AI provider is configured.

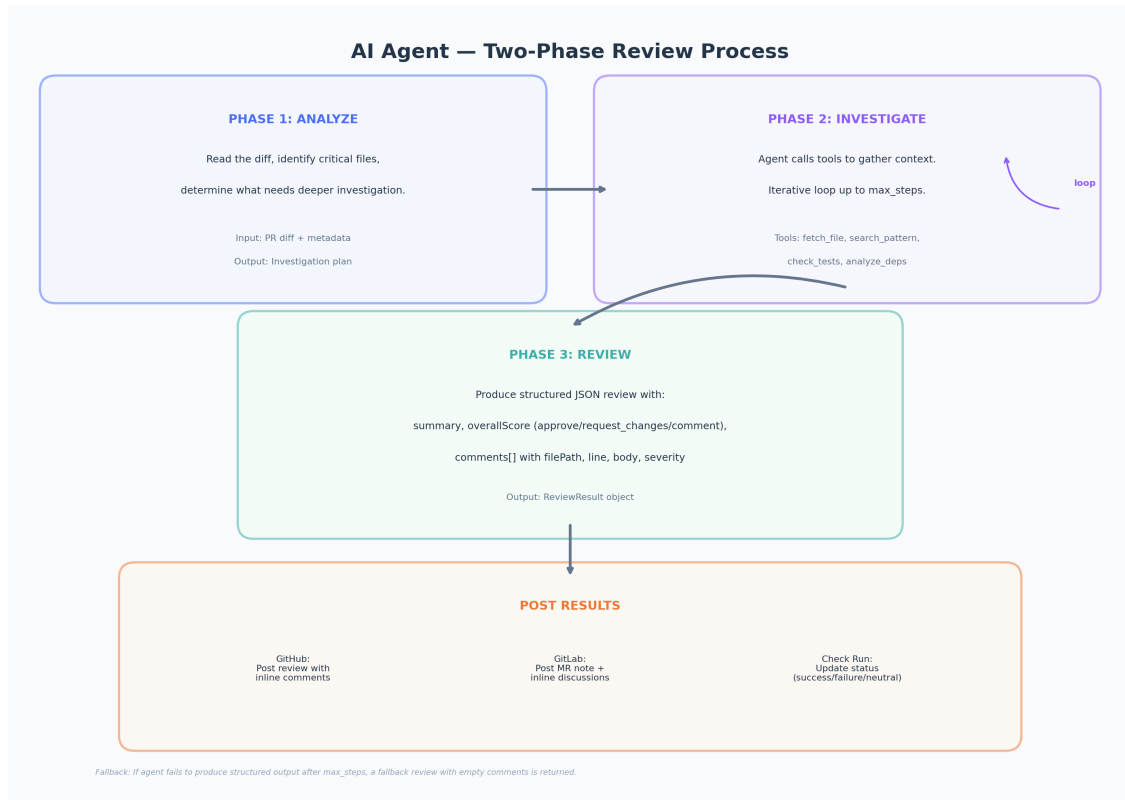


Figure 2: Two-phase AI agent review process

3. API Reference

The application exposes REST API routes for both internal dashboard consumption and external webhook integration. All API routes are implemented as Next.js API Route Handlers, which are server-side functions that run in the Node.js runtime. The following subsections document every endpoint, its request/response format, and its error handling behavior.

3.1 GitHub Webhook (POST /api/webhook)

This is the primary entry point for GitHub events. It handles five event types: `installation`, `installation_repositories`, `pull_request`, `issue_comment`, and `pull_request_review_comment`. The handler begins by reading the raw request body and verifying the HMAC-SHA256 signature if a webhook secret is configured. Signature verification uses `crypto.timingSafeEqual` to prevent timing attacks. If the signature does not match, a 401 response is returned immediately.

If no webhook secret is configured, signature verification is skipped (useful for development but not recommended for production).

For **pull_request** events with action "opened" or "synchronize", the handler extracts the PR details and kicks off an asynchronous review process. The review runs in a fire-and-forget promise chain (`processReview().catch(console.error)`), meaning the webhook response is returned immediately with a 200 status while the review continues in the background. This is essential because GitHub webhooks have a 10-second timeout, and AI reviews typically take 15-60 seconds.

For **issue_comment** events with action "created" on a PR (not a regular issue), the handler parses the comment body for slash commands (`/review`, `/recheck`, `/check`, `/re-review`). If a command is detected, it posts an acknowledgment comment, fetches the current PR context, and triggers a re-review with the `isReReview` flag set to true. The command arguments are parsed to extract focus files and questions. If the arguments look like a file path (containing `/` or common extensions), they are treated as a `focusFile`; otherwise, they are treated as a `focusQuestion`.

For **pull_request_review_comment** events (inline review comment replies), the handler applies the same slash command parsing but automatically uses the comment file path as the focus file. For **check_run** events with action "rerequested", the handler re-triggers the review for the associated PR.



Figure 3: Webhook event processing flow

3.2 GitLab Webhook (POST /api/webhook/gitlab)

The GitLab webhook handler processes Merge Request Hook events (for new and updated MRs) and Note Hook events (for comment-triggered re-reviews). Authentication is via the X-GitLab-Token header, which is compared against the configured webhook secret. The handler extracts the project path_with_namespace and splits it into owner and repo components. For self-hosted GitLab instances, the host URL is derived from the project web_url. The review is processed asynchronously, mirroring the GitHub handler pattern.

3.3 Configuration API (GET/POST /api/config)

The GET endpoint returns all configuration values, with sensitive values (tokens, keys, secrets) masked using a show-first-4-and-last-4 pattern. The response includes boolean flags indicating which integrations are configured (hasToken, hasGitHubApp, hasGitLabToken, etc.) and the blockMerge status. The POST endpoint accepts a key-value pair and upserts it into the AppConfig table. Only whitelisted keys are accepted: github_token, webhook_secret, github_app_id,

github_app_private_key, gitlab_token, gitlab_host, gitlab_webhook_secret, ai_provider, ai_model, ai_api_key, ai_base_url, ai_temperature, ai_max_steps, and block_merge. Any other key returns a 400 error.

3.4 Reviews API

GET /api/reviews returns a paginated list of reviews with optional status filtering. Each review includes the repository full name and comment count. Pagination uses page/limit parameters with a default of 10 items per page. **GET /api/reviews/[id]** returns a single review with all its comments and repository details. **DELETE /api/reviews/[id]** removes a review and its associated comments (cascading delete). **POST /api/reviews/trigger** manually triggers a review for a given owner/repo/PR number, supporting both GitHub and GitLab platforms.

3.5 API Summary Table

Endpoint	Method	Purpose	Auth
/api/webhook	POST	GitHub webhook receiver	HMAC-SHA256 signature
/api/webhook/gitlab	POST	GitLab webhook receiver	X-GitLab-Token header
/api/config	GET	Read all configuration	None (internal)
/api/config	POST	Update a config value	None (internal)
/api/reviews	GET	List reviews (paginated)	None (internal)
/api/reviews/[id]	GET	Get review detail	None (internal)
/api/reviews/[id]	DELETE	Delete a review	None (internal)
/api/reviews/trigger	POST	Manually trigger review	None (internal)

4. Database Schema

The database schema uses Prisma ORM with SQLite as the storage engine. SQLite was chosen for its zero-configuration deployment model, single-file storage, and excellent read performance for the workload pattern of this application (predominantly reads with occasional writes during review processing). The schema consists of four models that capture the complete data domain of the application.

4.1 Entity-Relationship Diagram

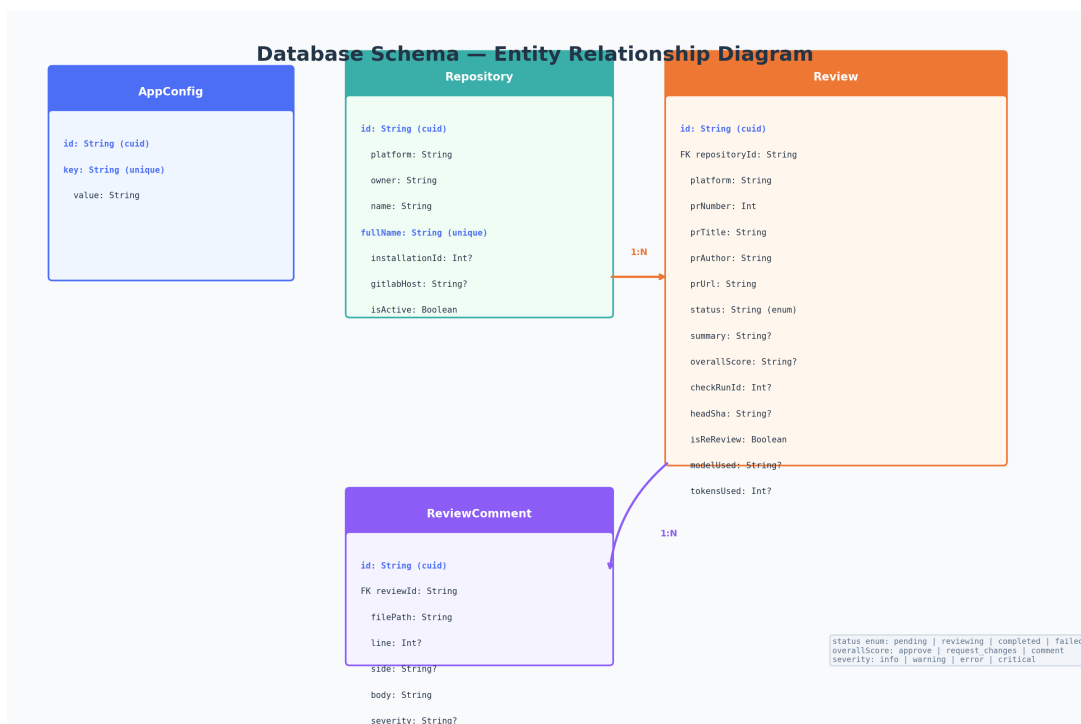


Figure 4: Database entity-relationship diagram

4.2 Model Definitions

AppConfig

The AppConfig model stores all application configuration as key-value pairs. This design was chosen over individual environment variables for three reasons. First, it allows the dashboard UI to read and modify configuration without requiring server restarts. Second, it provides a single source of truth that is shared across all API routes. Third, it supports the masked display of sensitive values (tokens, keys) in the UI without requiring separate secret management infrastructure. The key field has a unique constraint, enforced by Prisma at the database level, preventing duplicate configuration entries.

Field	Type	Constraints	Example Values
id	String	@id @default(cuid())	clxyz123...
key	String	@unique	github_token, ai_provider, block_merge
value	String	None	ghp_xxx, z-ai, false

Repository

The Repository model tracks installed repositories across both GitHub and GitLab. The platform field distinguishes between GitHub and GitLab repositories. The installationId field stores the GitHub App installation ID, which is required for generating installation-scoped access tokens. The gitlabHost field stores the base URL for self-hosted GitLab instances. The fullName field (format: "owner/name") has a unique constraint and serves as the natural key for repository lookups. The isActive flag supports soft deletion when a GitHub App installation is removed.

Field	Type	Constraints	Purpose
id	String	@id @default(cuid())	Internal primary key
platform	String	@default("github")	Platform identifier (github/gitlab)
owner	String	None	Repository owner or namespace
name	String	None	Repository name
fullName	String	@unique	Composite key: owner/name
installationId	Int?	None	GitHub App installation ID
gitlabHost	String?	None	Self-hosted GitLab URL
isActive	Boolean	@default(true)	Soft delete flag

Review

The Review model is the central data entity, capturing the complete lifecycle of a code review. The status field tracks the review state machine: pending (created but not started), reviewing (AI agent is processing), completed (review finished successfully), or failed (an error occurred during processing). The overallScore field stores the AI agent verdict (approve, request_changes, or comment). The

checkRunId and headSha fields are used together to update GitHub Check Runs. The agentSteps field stores a JSON-serialized array of AgentStep objects, providing a full audit trail of the AI agent decision process. The isReReview flag distinguishes between initial reviews and those triggered by slash commands, which is important for the comment reply logic.

Field	Type	Constraints	Purpose
id	String	@id @default(cuid())	Internal primary key
repositoryId	String	FK -> Repository	Parent repository
platform	String	@default("github")	Platform of the PR/MR
prNumber	Int	None	PR/MR number
prTitle	String	None	PR/MR title
prAuthor	String	None	PR/MR author username
prUrl	String	None	URL to the PR/MR
status	String	@default("pending")	Review state
summary	String?	None	AI-generated review summary
overallScore	String?	None	approve/request_changes/comment
checkRunId	Int?	None	GitHub Check Run ID
headSha	String?	None	HEAD commit SHA
agentSteps	String?	None	JSON array of agent steps
isReReview	Boolean	@default(false)	Slash command triggered
modelUsed	String?	None	AI model identifier

tokensUsed	Int?	None	Approximate token count
------------	------	------	-------------------------

ReviewComment

The ReviewComment model stores individual inline findings from a review. Each comment belongs to a Review (cascading delete) and includes the file path, line number, diff side, comment body, and severity classification. The severity field uses a four-level scale: info (suggestions and minor observations), warning (best practice violations and style issues), error (bugs and logic errors), and critical (security vulnerabilities and data loss risks). This severity system drives both the visual presentation in the GitHub/GitLab UI (via severity-prefixed comment bodies) and the Check Run annotation levels (info maps to "notice", warning maps to "warning", error and critical map to "failure").

5. Interactive Slash Commands

The slash command system enables developers to interact with the AI review agent directly within PR comment threads, similar to how Claude Code and Gemini review agents work. This feature transforms the agent from a passive review tool into an active participant in the code review conversation. Developers can request re-reviews, focus on specific files, ask questions about code, and more, all without leaving the GitHub/GitLab interface.

5.1 Supported Commands

Command	Arguments	Behavior
/review	None	Trigger a fresh full review of the entire PR
/recheck	None	Alias for /review (triggers re-review)
/check	None	Alias for /review (triggers re-review)
/re-review	None	Alias for /review (triggers re-review)
/review path/to/file	File path	Re-review focusing on a specific file
/check question text	Text	Re-review with a specific question for the agent
/review src/app.ts Why...	Path + question	Focus on a file with a specific question

5.2 Command Processing Pipeline

When a developer types a slash command in a PR comment, the following processing pipeline executes. First, the GitHub webhook fires an `issue_comment` event (for general PR comments) or a `pull_request_review_comment` event (for inline review replies). The webhook handler checks that the action is "created" (ignoring edited and deleted comments) and that the comment is on a PR (not a regular issue, determined by the presence of the `pull_request` field in the issue object).

Second, the handler parses the comment body by trimming whitespace, converting to lowercase, and checking if it starts with any of the supported command strings. If a command is detected, the remaining text after the command is extracted as arguments. Third, the handler posts an acknowledgment comment to the PR, confirming that the re-review has been triggered and showing the detected focus parameters. This immediate feedback is important for developer experience, as the actual review may take 15-60 seconds to complete.

Fourth, the handler fetches the current PR information (to get the `headSha` for Check Runs) and triggers the review process with `isReReview=true`. The argument parsing logic attempts to detect file paths (strings containing `/` or common code extensions like `.ts`, `.js`, `.py`, `.go`, `.rs`) and treats them as `focusFile` parameters, with any remaining text as `focusQuestion`. If no file path is detected, the entire argument string is treated as `focusQuestion`.

Fifth, after the review completes, the `processReview` function posts the review with inline comments and attempts to reply to the original comment thread. If the reply fails (for example, the review thread is on an outdated diff), it falls back to posting a general PR comment. This ensures the developer always receives a response, even if the exact reply mechanism is unavailable.

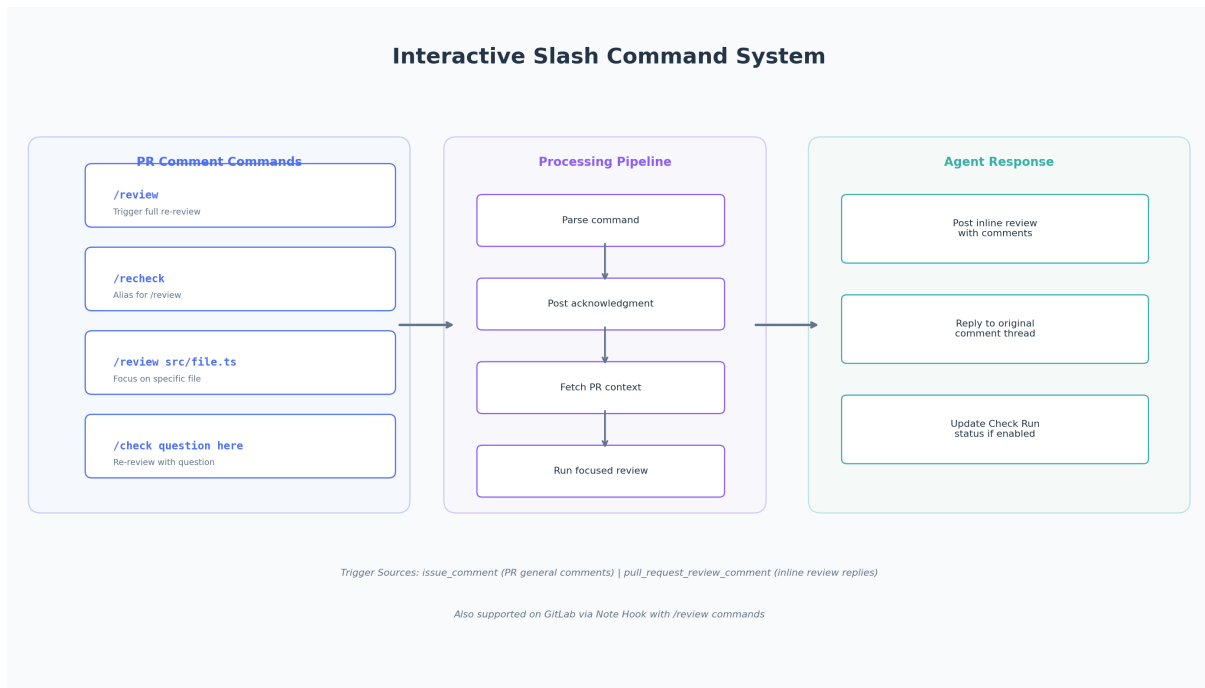


Figure 5: Interactive slash command processing pipeline

5.3 GitLab Slash Commands

GitLab supports equivalent slash command functionality through the Note Hook webhook event. When a developer types a slash command in a Merge Request comment, GitLab fires a Note Hook event with `noteable_type` set to "MergeRequest". The handler checks the note body for supported commands and triggers a re-review. The GitLab implementation currently supports `/review`, `/recheck`, `/check`, and `/re-review` commands. Focus file and question parsing is not yet implemented for GitLab, but the architecture supports adding this feature by extending the Note Hook handler.

6. Merge Blocking Configuration

Merge blocking is one of the most impactful features of the AI PR Reviewer, but it is also the feature that requires the most careful handling. Blocking merges by default would impose the tool opinion on every team, regardless of their workflow maturity. The design decision to make merge blocking OFF by default is rooted in the principle that the tool should be advisory first, enforcing second. Teams should experience the value of AI review before committing to merge-blocking enforcement.

6.1 How Merge Blocking Works

Merge blocking is implemented through GitHub Check Runs. When a review is triggered for a PR, and merge blocking is enabled, the `processReview` function creates a Check Run with status `"in_progress"`. This Check Run appears in the GitHub PR checks UI. When the review completes, the Check Run is updated with a conclusion based on the review result. If the overall score is `"approve"`, the conclusion is `"success"` (green check). If the score is `"request_changes"`, the conclusion is `"failure"` (red X). If the score is `"comment"`, the conclusion is `"neutral"` (gray circle).

For the Check Run to actually block merging, the repository administrator must configure the "AI Code Review" check as a required status check in the repository branch protection rules. This is a separate, manual step that is not automated by the application. This two-step approach (enable merge blocking in the app settings, then require the check in branch protection) provides an important safety net: even if merge blocking is accidentally enabled in the app, it will not block merges until the branch protection rule is also configured.

6.2 Configuration States

block_merge Setting	Review Result	Check Run Conclusion	Blocks Merge?
false (default)	approve	success	No (but shows green check)
false (default)	request_changes	neutral	No (advisory only)
false (default)	comment	neutral	No (advisory only)
true	approve	success	No (green check)
true	request_changes	failure	Yes (if required in branch protection)
true	comment	neutral	No (informational)

6.3 Check Run Output

When merge blocking is disabled (the default), the Check Run output includes the text `"Merge blocking is disabled - this check is advisory only."` appended to the review summary. This makes it immediately clear to developers and repository administrators that the check is informational and not blocking merges. When merge blocking is enabled, the output contains only the review summary, as the conclusion itself communicates the enforcement status.

The Check Run output also includes annotations for each review comment that has a line number. These annotations appear directly in the GitHub Files Changed

tab, showing the severity and message of each finding. Annotations are limited to 50 per Check Run (a GitHub API limit), so only the first 50 inline comments are included as annotations. The annotation level mapping follows the severity scale: critical and error map to "failure" (red), warning maps to "warning" (yellow), and info maps to "notice" (blue).

7. Review Processing Pipeline

The review processing pipeline is the core data flow of the application, transforming a webhook event into a completed code review with inline comments on the source platform. This section traces the complete lifecycle of a review, from webhook receipt to result posting, explaining every step and its error handling behavior.

7.1 Pipeline Stages

Stage 1 - Webhook Receipt: The webhook handler receives the HTTP POST, verifies the signature, and parses the event payload. If signature verification fails, a 401 is returned. If the event type is not one of the supported types, a 200 with message "Event ignored" is returned. This fast-reject pattern ensures that the application does not waste resources processing irrelevant events.

Stage 2 - Repository Registration: The `upsertRepository` function is called to ensure the repository exists in the database with the correct installation ID. This is idempotent: if the repository already exists, only the `installationId` and `isActive` fields are updated.

Stage 3 - Review Record Creation: A new Review record is created with status "reviewing". The review ID is stored for use in subsequent stages. The `headSha` is stored for Check Run creation.

Stage 4 - Check Run Initiation: If merge blocking is enabled, a Check Run is created with status "in_progress". The Check Run ID is stored on the Review record for later updates. If Check Run creation fails, the error is logged but does not block the review. This is a non-fatal error because the review can still complete and post comments without the Check Run.

Stage 5 - Data Retrieval: The PR diff and metadata are fetched concurrently using `Promise.all` for optimal performance. The diff is the primary input to the review engine. The metadata provides context (title, author, description, branch names, change statistics) that helps the AI agent understand the intent of the changes.

Stage 6 - AI Review Execution: The reviewPR function is called with the diff, metadata, and tool context. This is the longest stage, typically taking 15-60 seconds depending on the size of the diff and the AI model response time. The function returns a ReviewResult object containing the summary, overall score, comments, agent steps, and model information.

Stage 7 - Database Update: The Review record is updated with the completed status, summary, overall score, agent steps, model used, and token count. Review comments are created in batch using createMany for efficiency.

Stage 8 - Result Posting: The review is posted back to GitHub or GitLab. For GitHub, this creates a Pull Request Review with inline comments and an overall verdict. For GitLab, this creates an MR note with the summary and individual inline discussions for each comment. If posting fails, the error is logged but the review is still marked as completed in the database, ensuring no data loss.

Stage 9 - Check Run Finalization: If a Check Run was created in Stage 4, it is updated with the final conclusion and output. Annotations are added for each inline comment. If the update fails, the error is logged but does not affect the review status.

Stage 10 - Comment Reply: If this was a re-review triggered by a slash command, a reply is posted in the original comment thread. If the reply fails, a fallback general PR comment is posted. This ensures the developer who triggered the re-review always gets a direct response.

7.2 Error Handling Strategy

The review pipeline uses a layered error handling strategy. At the outermost layer, the processReview function wraps the entire pipeline in a try/catch block. If any stage fails, the Review record is updated with status "failed" and the error message is stored as the summary. If a Check Run was created, it is updated with conclusion "neutral" and a summary indicating the review failed. This ensures that even failed reviews leave a trace in the database and the GitHub Checks UI, rather than silently disappearing.

At the inner layer, individual stages have their own try/catch blocks for non-fatal errors. Check Run creation failure, review posting failure, and comment reply failure are all non-fatal: they are logged but do not prevent subsequent stages from executing. Only the core review processing (diff fetching and AI review execution) are treated as fatal errors that terminate the pipeline. This design maximizes the chances of a useful result being produced, even when parts of the system are experiencing issues.

8. Dashboard UI

The dashboard is a single-page React application built with shadcn/ui components and Framer Motion animations. It provides three main sections: Dashboard, Manual Review, and Settings. The UI is fully responsive and supports dark mode through the next-themes integration in the shadcn/ui component library.

8.1 Dashboard Tab

The Dashboard tab displays four summary stat cards (Total Reviews, Approved, Changes Requested, Active Reviews) at the top, followed by a filterable, paginated table of recent reviews. Each review row shows the PR number, title, author, repository, status badge, score badge, comment count, and date. Clicking a row opens a detail sheet that shows the full review summary, all inline comments with severity badges, and the agent step audit trail. The table supports status filtering (All, Pending, Reviewing, Completed, Failed) and pagination with configurable page size. Active reviews (pending or reviewing status) trigger automatic refresh every 5 seconds, providing near-real-time progress updates.

8.2 Manual Review Tab

The Manual Review tab provides a simple form to trigger reviews for any public or accessible repository. The user enters the repository owner, repository name, and PR number. Upon submission, the trigger API is called, and a result card shows the review ID and a link to the Dashboard. The tab is useful for testing the review pipeline, reviewing PRs from repositories where webhook integration is not yet configured, or re-triggering failed reviews.

8.3 Settings Tab

The Settings tab is organized into four cards: GitHub App Configuration, GitLab Configuration, AI Configuration, and Merge Protection. Each setting has an input field and a Save button that calls the config API. Sensitive values (tokens, keys, secrets) are masked when displayed. The AI Configuration card supports switching between z-ai and openai-compatible providers, with the base URL defaulting to the Oencode API endpoint (<https://api.oencode.com/v1>). The Merge Protection toggle controls the block_merge configuration, which defaults to OFF (false). The current state of each setting is loaded from the config API on mount, and the UI reflects the actual stored values.

9. Deployment Guide

This section provides a complete, step-by-step deployment guide for running the AI PR Reviewer in a production environment. The deployment uses Caddy as a reverse proxy with automatic HTTPS, and the application runs as a standalone Next.js server.

9.1 Prerequisites

- Node.js 18+ or Bun runtime
- A domain name pointing to your server (for HTTPS via Caddy)
- A GitHub account with permission to create GitHub Apps
- An AI API key (z-ai-web-dev-sdk is included, or use Oencode/OpenAI-compatible endpoint)

9.2 GitHub App Setup

Navigate to GitHub Settings > Developer settings > GitHub Apps and click "New GitHub App". Configure the following settings:

Setting	Value	Notes
App Name	AI PR Reviewer (your-org)	Must be unique across GitHub
Homepage URL	https://your-domain.com	Your deployment URL
Webhook URL	https://your-domain.com/api/webhook	Must be HTTPS
Webhook Secret	Generate a random string	Store in app settings
Repository Permissions	Pull Requests: Read/Write, Contents: Read, Checks: Read/Write, Issues: Read/Write	Required for review and check runs
Subscribe to Events	Pull request, Installation, Check run, Issue comment	Required for all features
Where can this app be installed?	Any account	Or restrict to your org

After creating the app, generate a private key and download the PEM file. Note the App ID from the app settings page. Both the App ID and private key will be entered in the dashboard Settings tab.

9.3 Application Deployment

Step 1: Clone or unzip the project and install dependencies with `bun install`. **Step 2:** Configure the environment by creating a `.env` file with `DATABASE_URL=file:./dev.db`. **Step 3:** Run database migrations with `bunx prisma migrate deploy`. **Step 4:** Build the application with `bun run build`. **Step 5:** Start the production server with `bun run start`. **Step 6:** Configure Caddy as a reverse proxy to `localhost:3000`. The included Caddyfile provides a basic configuration that also supports a dynamic port query parameter for development purposes.

9.4 Configuration Checklist

After deployment, visit the dashboard Settings tab and configure the following items in order: (1) GitHub App ID and Private Key, or a GitHub Personal Access Token as an alternative. (2) Webhook Secret (must match the value set in the GitHub App settings). (3) AI Provider and Model (z-ai is zero-configuration; for Oencode, enter the API key and base URL). (4) Merge Protection toggle (leave OFF unless you have configured branch protection rules). (5) Optional: GitLab Token and Host URL if you need GitLab MR review support.

10. Security Considerations

Security is a critical concern for any system that handles source code and has write access to GitHub and GitLab repositories. This section documents the security measures implemented in the AI PR Reviewer and identifies areas that require attention during deployment.

10.1 Webhook Signature Verification

The application verifies the HMAC-SHA256 signature on all incoming GitHub webhooks. The signature is computed using the raw request body and the configured webhook secret, then compared using `crypto.timingSafeEqual` to prevent timing attacks. If the signature does not match, the request is rejected with a 401 status. For GitLab, the `X-GitLab-Token` header is compared against the configured token. If a webhook secret is not configured, signature verification is skipped entirely. This is acceptable for development but must be configured for production deployments.

10.2 Credential Storage

All credentials (GitHub tokens, private keys, GitLab tokens, AI API keys) are stored in the SQLite database via the `AppConfig` table. The configuration API masks sensitive values when returning them in GET responses, using a

show-first-4-and-last-4 pattern. However, the database file itself is not encrypted, which means that anyone with filesystem access to the server can read all stored credentials. For production deployments with heightened security requirements, consider using environment variables for sensitive values or integrating with a secrets management service. The POST `/api/config` endpoint only accepts whitelisted configuration keys, preventing arbitrary data injection.

10.3 GitHub App Permissions

The GitHub App operates with the minimum permissions required for its functionality. Pull request read/write access is needed to post reviews and comments. Contents read access is needed for the `fetch_file` agent tool. Checks read/write access is needed for Check Run creation and updates. Issue comment read/write access is needed for the slash command system. The app does not request repository admin access, delete permissions, or any other elevated privileges. The principle of least privilege is maintained throughout the permission model.

10.4 AI Prompt Injection

The AI review engine processes untrusted input (PR diffs and descriptions) and includes it in prompts sent to the AI model. There is a theoretical risk of prompt injection, where a malicious PR author crafts diff content that causes the AI model to produce unintended output. The system mitigates this risk through several mechanisms: the review output is constrained to a specific JSON schema, the system prompt explicitly instructs the model to only produce review output, and the `parseReviewFromContent` function only accepts JSON blocks that match the expected schema. However, no AI system is fully immune to prompt injection, and this risk should be acknowledged in security reviews.

10.5 Rate Limiting and Abuse Prevention

The current implementation does not include built-in rate limiting for webhook endpoints. In a production deployment, rate limiting should be implemented at the reverse proxy level (Caddy supports rate limiting via plugins) or through middleware. Without rate limiting, a malicious actor could flood the webhook endpoint with events, causing excessive AI API usage and costs. The GitHub and GitLab platforms themselves impose rate limits on webhook delivery, which provides some natural protection, but this should not be relied upon as the sole defense.